Inclusive Talent Navigator

Prototype Documentation

Repository: `ByteForge-Inclusive-Talent-Navigator`
Prototype version: 1.0.0
Status: SAP Hackfest 2026 Round 2 prototype
Last documented: November 2026

---

1. Executive summary

Inclusive Talent Navigator is an internal talent mobility prototype that helps employees understand their transferable skills and helps HR identify internal opportunities through a more evidence-based and reviewable process.

The prototype connects employee-owned profile evidence with internal job roles, skill taxonomy data, market-demand signals, learning resources, and fairness review indicators. It does not make an automated hiring decision. Instead, it provides structured recommendations and explanations that employees and HR partners can review and act on.

The product has two role-specific experiences:

- **Employee workspace:** manage profile documents, review extracted skills, explore internal role matches, inspect fairness information, view market intelligence, and follow a learning pathway.
- **HR partner workspace:** manage roles and required skills, run internal opportunity matching, inspect match evidence, run bias reviews, record decisions, and inspect HR activity.

The application is intentionally designed around explicit evidence, human confirmation, auditable decisions, and replaceable domain agents rather than opaque end-to-end automation.

2. Problem and intended users

Problem

Traditional talent processes can hide employee capabilities because:

- skills are recorded inconsistently across resumes, projects, and systems;
- related technologies may be treated as interchangeable without evidence;
- employees may not know which internal roles or learning resources fit next;
- HR decisions can be difficult to explain or review after the fact; and
- job descriptions and matching workflows can carry language or process bias.

Intended users

User	Primary needs	Prototype workspace
Employee	Build a trustworthy skills profile, discover roles, and plan development	Employee workspace
HR partner	Configure roles, discover internal candidates, and review fairness signals	HR workspace
HR reviewer/administrator	Inspect activity and decision history	HR audit views

3. Product principles

1. **Evidence before inference.** Extracted skills retain source context and evidence snippets for employee review.
2. **Employee confirmation.** Extracted skills are presented as drafts until the employee confirms, removes, or adds them.
3. **Exact matching.** Matching uses normalized skill names and evidence phrases; it does not silently assume that related technologies are equivalent.
4. **Human-in-the-loop fairness.** Bias flags are review indicators, not automated findings of discrimination.
5. **Auditable actions.** Agent runs, skill changes, matching activity, bias reviews, and HR decisions are recorded in database audit tables.
6. **Separation of responsibilities.** HTTP routes, orchestration, database access, domain reasoning, and MCP transport are kept in separate layers.
7. **No external inference dependency.** The current prototype uses local domain logic behind MCP tools and does not call an external LLM or hosted inference provider.

4. Prototype capabilities

Employee capabilities

- Employee login and logout using an HTTP-only cookie session.
- Mandatory first-login password change when the account requires it.
- Employee dashboard with profile and readiness information.
- Upload, list, and delete profile documents.
- Extract skills from uploaded documents.
- Review pending skills and confirm the final skill set.
- View inferred proficiency, evidence confidence, and evidence snippets.
- View internal role matching results and match explanations.
- View employee-facing fairness review information.
- View market demand and trend signals for confirmed skills.
- View a learning pathway based on persisted matching results and learning courses.
- View employee audit history.

HR capabilities

- HR partner login and logout.

- HR dashboard with operational and readiness indicators.
- List, create, update, activate, and deactivate job roles where permitted.
- Add and remove required skills and weights for roles.
- Run inclusive matching across eligible employees and active roles.
- Filter and inspect persisted matching results.
- Run bias review for an employee, match set, or set of roles.
- Filter and inspect bias flags.
- Review flags with status and comments.
- Record a decision on a matching result, including an override reason where appropriate.
- View HR activity audit logs.

Health and operational capabilities

- Root service health response.
- Application health endpoint.
- Database connectivity health endpoint.
- MySQL and SAP HANA database adapter selection through configuration.
- Explicit errors for unsupported document extraction situations, including OCR-dependent scanned documents when OCR is unavailable.

5. User experience and navigation

Employee routes

Route	Screen	Purpose
`/`	Login	Employee or HR entry point
`/change-password`	Change password	Complete required first-login password change
`/dashboard`	Employee dashboard	Profile summary and next action
`/skills`	Skills discovery	Upload documents, extract, review, and confirm skills
`/matching`	Role matching	Review internal opportunity matches
`/bias-audit`	Fairness review	Review employee-facing fairness information
`/learning`	Learning pathway	Explore recommended development courses
`/audit-log`	Employee audit log	Review employee-visible activity

HR routes

Route	Screen	Purpose
`/hr/dashboard`	HR dashboard	Operational overview
`/hr/jobs`	Job roles	Maintain roles and required skills
`/hr/matching`	HR matching	Run and review internal matching
`/hr/bias-audit`	HR bias audit	Review flags and record decisions

Route	Screen	Purpose
`/hr/audit-log`	HR audit log	Review HR activity

The frontend is a React application built with Vite. Employee and HR routes use separate layouts, navigation sidebars, authentication contexts, and protected-route components.

6. End-to-end workflows

6.1 Employee onboarding and skills discovery

1. The employee signs in.
2. If required, the employee changes the temporary password.
3. The employee uploads supported profile documents.
4. The backend validates ownership and extracts text from each document.
5. The extracted documents are rebuilt into a single `PROFILE_TEXT` value with document boundaries and source metadata.
6. The employee starts skill extraction.
7. The Skills Discovery Agent calls the Skills MCP server.
8. Skills are normalized against the database-backed taxonomy and aliases.
9. Pending skills, inferred proficiency, confidence, and evidence are shown.
10. The employee confirms, removes, or adds skills.
11. Manual additions are normalized through MCP before final persistence.
12. The employee becomes eligible for matching according to the persisted readiness state.

Supported document formats include PDF, DOCX, TXT, RTF, XLSX, PPTX, and common image files at the extraction layer. The current documented product workflow focuses on PDF and DOCX uploads. Scanned PDFs and image uploads require Tesseract OCR; when OCR is not configured, the backend returns an explicit error rather than saving an empty profile.

6.2 Internal opportunity matching

1. An authenticated HR partner selects employees and/or job roles, or runs the configured matching scope.
2. The matching controller loads eligible employees, active roles, required skills, and confirmed employee skills.
3. The Opportunity Matching Agent starts the Matching MCP server over stdio.
4. The `run_matching` tool calculates weighted skill coverage and experience-fit warnings.
5. The agent validates the structured MCP response.
6. Matching results and a central audit event are persisted.
7. The HR interface reloads persisted results and displays the breakdown.
8. HR can inspect a result and record a proceed, hold, reject, or override decision as supported by the request schema.

Matching is a recommendation and prioritization aid. It is not a hiring decision and should not be used without human review.

6.3 Bias review

1. HR starts a bias review for an employee, matching results, and/or roles.
2. The Bias Review Agent loads role criteria and relevant match indicators.
3. The Bias MCP server evaluates job wording and matching review indicators.
4. Structured flags are persisted in `BIAS_AUDIT_FLAGS`.
5. HR filters and opens individual flags.
6. HR records a review status and comments.
7. Decisions and review activity remain available through audit views.

Bias flags are signals for review. They are not automatic determinations that discrimination occurred.

6.4 Market intelligence

1. The employee opens Market Intelligence through the dashboard/navigation.
2. Confirmed skills are loaded from the database.
3. The Market Intelligence Agent invokes the Market MCP server.
4. The MCP tool compares skills with active `MARKET_DEMAND` rows.
5. The response includes demand level, trend, internal openings, and notes.
6. The model records a central audit event.

The current data is database-backed prototype data. The SAP Learning Hub integration is intentionally not enabled until an approved SuccessFactors Learning tenant and API credentials are available.

6.5 Career development

1. The employee opens the Learning Pathway page.
2. Persisted top-role matching results are converted into the agent input shape.
3. The Career Development Agent invokes the Learning MCP server.
4. The MCP tool selects active courses relevant to target skills.
5. The response includes course title, target skill, duration, and SAP Learning Hub URL.
6. The model records a central audit event.

7. Technical architecture

```
React/Vite frontend
        |
        | HTTP requests with credentials included
        v
FastAPI routes
        |
        | authentication, request validation, response schemas
        v
Controllers
        |
        | orchestration
        v
Models <--------------------> MySQL or SAP HANA
        |
        | domain inputs/outputs
        v
Agents
```

```
          |
          | MCP client over stdio
          v
  Domain MCP servers
          |
          v
  Structured domain results
```

Layer responsibilities

Layer	Responsibility	Not responsible for
Frontend	Navigation, forms, loading/error states, rendering	Database access or domain calculations
Routes	HTTP endpoints, dependencies, request/response schemas	SQL or domain calculations
Controllers	Simple request orchestration	Direct SQL or agent transport details
Models	Queries, persistence, audit writes	UI concerns or domain transport
Agents	MCP client calls, response validation, public domain entry points	Database connections or local audit files
MCP servers	Specialized structured domain calculations	Persistence or HTTP routing
Database adapters	MySQL/HANA connectivity	Product-level decisions

Agent and MCP map

Domain	Agent entry point	MCP server	Tool
Skills Discovery	`SkillsDiscoveryAgent`	`skill_mcp_server.py`	`extract_skills`, `normalize_skills`
Opportunity Matching	`run_inclusive_matching`	`matching_mcp_server.py`	`run_matching`
Bias Review	`run_bias_audit`	`bias_mcp_server.py`	`run_bias_audit`
Market Intelligence	`run_market_intelligence`	`market_mcp_server.py`	`analyze_market_demand`
Career Development	`run_learning_pathway`	`learning_mcp_server.py`	`generate_learning_pathway`

The public agent entry points remain compatible with their controllers. Each MCP server invokes an internal domain core so that specialized calculations remain in the MCP boundary while transport and response handling remain in the agent.

8. Backend API reference

All protected requests use cookie credentials. Employee endpoints use the `employee_token` cookie; HR endpoints use the `hr_token` cookie.

Health

Method	Endpoint	Auth	Purpose
`GET`	`/`	Public	Confirm the API is running
`GET`	`/api/health`	Public	Return service and configured database type
`GET`	`/api/health/database`	Public	Verify database connectivity and report database identity

Employee authentication and documents

Method	Endpoint	Auth	Purpose
`POST`	`/api/auth/login`	Public	Authenticate an employee
`POST`	`/api/auth/logout`	Employee cookie	Clear employee session
`POST`	`/api/auth/change-password`	Employee cookie	Change a required temporary password
`GET`	`/api/auth/me`	Employee cookie	Return current employee profile
`POST`	`/api/documents/upload`	Employee cookie	Upload one or more employee documents
`GET`	`/api/documents/{employee_id}`	Employee cookie	List owned documents
`DELETE`	`/api/documents/{employee_id}/{document_id}`	Employee cookie	Delete an owned document

Employee domain features

Method	Endpoint	Auth	Purpose
`POST`	`/api/skills/extract`	Active employee	Extract and normalize skills
`POST`	`/api/skills/confirm`	Active employee	Persist employee-confirmed skills
`GET`	`/api/skills/{employee_id}`	Active employee	Get the employee skill state
`GET`	`/api/skills/audit/logs`	Active employee	Get employee skill audit events
`GET`	`/api/market/intelligence`	Active employee	Get market demand signals
`GET`	`/api/learning/pathway`	Active employee	Get recommended learning pathway

HR authentication and role management

Method	Endpoint	Auth	Purpose
POST	/api/hr/login	Public	Authenticate an HR partner
POST	/api/hr/logout	HR cookie	Clear HR session
GET	/api/hr/me	HR cookie	Return current HR profile
GET	/api/hr/dashboard	HR cookie	Return HR operational summary
GET	/api/hr/jobs/	HR cookie	List job roles
POST	/api/hr/jobs/	HR cookie	Create a job role
GET	/api/hr/jobs/{job_role_id}	HR cookie	Get one job role
PUT	/api/hr/jobs/{job_role_id}	HR cookie	Update an editable role
PATCH	/api/hr/jobs/{job_role_id}/status	HR cookie	Activate/deactivate a role
POST	/api/hr/jobs/{job_role_id}/skills	HR cookie	Add required skills
DELETE	/api/hr/jobs/{job_role_id}/skills/{requirement_id}	HR cookie	Remove a required skill
GET	/api/hr/jobs/audit/logs	HR cookie	Get HR role-management activity

HR matching and fairness

Method	Endpoint	Auth	Purpose
POST	/api/hr/matching/run	HR cookie	Run inclusive internal matching
GET	/api/hr/matching/results	HR cookie	List results with optional employee/role filters
GET	/api/hr/matching/results/{match_id}	HR cookie	Get one match and its evidence

Method	Endpoint	Auth	Purpose
POST	/api/hr/bias/run	HR cookie	Run a bias review
GET	/api/hr/bias/flags	HR cookie	List flags with optional filters
GET	/api/hr/bias/flags/{flag_id}	HR cookie	Get flag details
PATCH	/api/hr/bias/flags/{flag_id}/review	HR cookie	Record flag review status/comments
GET	/api/hr/bias/decisions/{match_id}	HR cookie	List decisions for a match
POST	/api/hr/bias/decisions/{match_id}	HR cookie	Record an HR matching decision

9. Data and persistence model

The database adapter is selected with DB_TYPE and supports mysql and hana. Complete schema and seed scripts exist in Backend/database/mysql/ and Backend/database/hana/.

Core operational entities

The application reads and writes the following established operational entities:

- EMPLOYEES: employee identity, profile text, readiness, and account state.
- EMPLOYEE_DOCUMENTS: uploaded files, extracted text, type, and processing status.
- JOB_ROLES: internal role definitions and active/predefined status.
- Job-role skill requirements: normalized required skills and weights.
- MATCHING_RESULTS: persisted employee-to-role match outputs.
- BIAS_AUDIT_FLAGS: review indicators produced by bias review.
- HR matching decisions: human decisions and override context.
- AUDIT_LOGS: central cross-domain audit events.
- HR_AUDIT_LOGS: HR-specific activity history.

Agent-domain tables

Table	Purpose	Important fields
SKILL_TAXONOMY	Canonical skills and categories	SKILL_ID, CANONICAL_NAME, CATEGORY, IS_ACTIVE
SKILL_TAXONOMY_ALIASES	Alternate names used for normalization	SKILL_ID, ALIAS
EMPLOYEE_SKILL_PROGRESS	Skill evidence and progress metadata	employee, skill, proficiency, confidence, evidence count, validation

Table	Purpose	Important fields
`MARKET_DEMAND`	Demand and trend signals	skill, demand level, trend, openings, note
`LEARNING_COURSES`	Database-backed learning catalog	course, target skill, duration, SAP Learning Hub URL

Skill evidence lifecycle

```
Uploaded document
  -> extracted text
  -> candidate skill
  -> canonical skill / alias match
  -> pending employee review
  -> confirmed skill
  -> eligible matching input
```

Confirmed skills can carry inferred proficiency, evidence confidence, last evidence date, project evidence count, employee confirmation state, and manager-validation metadata.

Matching result interpretation

The matching output is intended to expose more than a single score. It can include:

- skill coverage;
- evidence strength;
- weighted required-skill contribution;
- experience-fit or experience-gap warnings;
- matched and missing skills; and
- review/audit metadata.

10. Security, privacy, and governance

Authentication and authorization

- Employee and HR authentication are separate.
- Sessions are represented by HTTP-only cookies.
- Cookie security, expiry, and JWT lifetime are configuration-driven.
- Employee routes force the authenticated employee ID rather than trusting a client-supplied identity.
- Employees cannot read or modify another employee's documents or skills.
- HR routes require the HR authentication dependency.
- Password-change gating prevents access to active employee features before a temporary password is changed.

Data handling

- Uploaded document processing is performed by the backend.
- Profile text is rebuilt with document boundaries and source metadata.
- Extraction failures are returned explicitly.

- Domain data is stored in database tables and initialization scripts rather than hard-coded inside agents.
- Central audit events are written to database audit tables; agents do not create local audit files.

Human oversight

- Employees review extracted skills before confirmation.
- HR reviews matching evidence and bias flags.
- Bias indicators do not automatically label an employer or role as discriminatory.
- HR decisions and override reasons are recorded for later inspection.

Prototype security considerations

Before production use, the system should receive a formal security and privacy review covering secret management, cookie settings in deployed environments, CSRF protection, rate limiting, upload malware scanning, file-size limits, retention/deletion policies, encryption, database least privilege, and tenant/organizational isolation.

11. Local development and operation

Prerequisites

- Python environment for the backend.
- Node.js environment for the frontend.
- MySQL or SAP HANA connection configured for the selected adapter.
- Tesseract OCR only if scanned PDFs or image-based extraction is required.

Backend configuration

The backend loads environment variables from `Backend/.env`. The important configuration includes:

Variable	Purpose
`DB_TYPE`	Select `mysql` or `hana`; defaults to `hana`
Database connection variables	Used by the selected adapter
`JWT_EXPIRE_MINUTES`	Cookie/JWT lifetime
`COOKIE_SECURE`	Enable secure cookies in HTTPS deployments
`TESSERACT_CMD`	Optional path to the Tesseract executable

Never commit `.env` files, passwords, tokens, or database credentials.

Start the backend on Windows

From the repository root:

```
Push-Location .\Backend
& .\.venv\Scripts\python.exe -m uvicorn main:app --reload
Pop-Location
```

Start the frontend

From `Frontend`:

```
npm install
npm run dev
```

The frontend defaults to `http://localhost:8000` for the backend. Set `VITE_BACKEND_URL` when the backend runs elsewhere.

Validation commands

Backend compilation:

```
& .\Backend\.venv\Scripts\python.exe -m compileall -q .\Backend
```

Focused MCP smoke test:

```
Push-Location .\Backend
& .\.venv\Scripts\python.exe -c "from agents.matching_agent import
run_inclusive_matching; print(run_inclusive_matching([], [], [], []))"
Pop-Location
```

Frontend build and lint:

```
Push-Location .\Frontend
npm run build
npm run lint
Pop-Location
```

12. Demonstration script

The following sequence demonstrates the complete prototype story:

1. Start the database, backend, and frontend.
2. Sign in as an employee and complete the password change if prompted.
3. Open **Skills** and upload `Backend/test_docs/sample_resume.pdf`.
4. Run extraction and review the evidence-backed pending skills.

5. Confirm the valid skills and inspect the updated dashboard readiness.
6. Open **Matching** and review the employee-facing opportunities.
7. Open **Market Intelligence** and compare confirmed skills with demand levels and trends.
8. Open **Learning Pathway** and inspect recommended courses.
9. Open **Audit Log** to show the recorded employee activity.
10. Sign out and sign in through the HR workspace.
11. Open **Job Roles** and review or create an editable role and skill requirements.
12. Open **HR Matching**, run matching, and inspect a match breakdown.
13. Open **Bias Audit**, run a review, inspect a flag, and record a review.
14. Record an HR matching decision and open the HR audit log.

13. Current limitations and future roadmap

Current prototype limitations

- Market demand and learning-course data are seeded prototype rows, not a live labor-market or SAP Learning Hub synchronization.
- OCR depends on a separately installed Tesseract executable.
- The application is demonstrated as a local/internal prototype and does not claim production deployment readiness.
- Matching and bias review are decision-support features, not autonomous employment decisions.
- The current data model and authentication flows require production hardening before use with sensitive workforce data.
- The prototype's supported role and taxonomy coverage is limited to the configured database seed data.

Recommended next steps

1. Complete security, privacy, accessibility, and responsible-AI reviews.
2. Add production secret management, upload scanning, retention controls, and comprehensive authorization tests.
3. Expand taxonomy governance, aliases, proficiency calibration, and evidence feedback loops.
4. Add automated test coverage for route authorization, extraction errors, agent/MCP contract validation, and database adapter parity.
5. Integrate approved SAP SuccessFactors Learning data when tenant access and credentials are available.
6. Replace seeded market signals with governed, timestamped sources and clearly label their provenance.
7. Add explainability and review analytics for matching outcomes over time.
8. Add accessibility and internationalization validation for the frontend.
9. Define operational monitoring, backup, disaster recovery, and support procedures.

14. Repository map

```
Backend/
  main.py                        FastAPI application and health endpoints
  agents/                        Agent entry points and MCP orchestration
  app/routes/                    HTTP route definitions
  app/controllers/               Request orchestration
  app/models/                    Database access and persistence
```

```
  app/schemas/                  Pydantic request/response schemas
  app/utils/                    Authentication, extraction, and audit helpers
  database/mysql/               MySQL schema and seed data
  database/hana/                SAP HANA schema and seed data
  mcp_server/                   Domain MCP tools
  scripts/                      Operational scripts
  test_docs/                    Sample document for demonstrations

Frontend/
  src/App.jsx                   Route tree and workspace layouts
  src/pages/                    Employee and HR screens
  src/components/               Shared and workspace-specific UI
  src/contexts/                 Employee and HR authentication state
  src/services/                 Backend API clients
```

For contributor-level implementation rules, see `AGENT_CONTEXT/architecture.md`, `AGENT_CONTEXT/development.md`, and `AGENT_CONTEXT/workflows.md`.